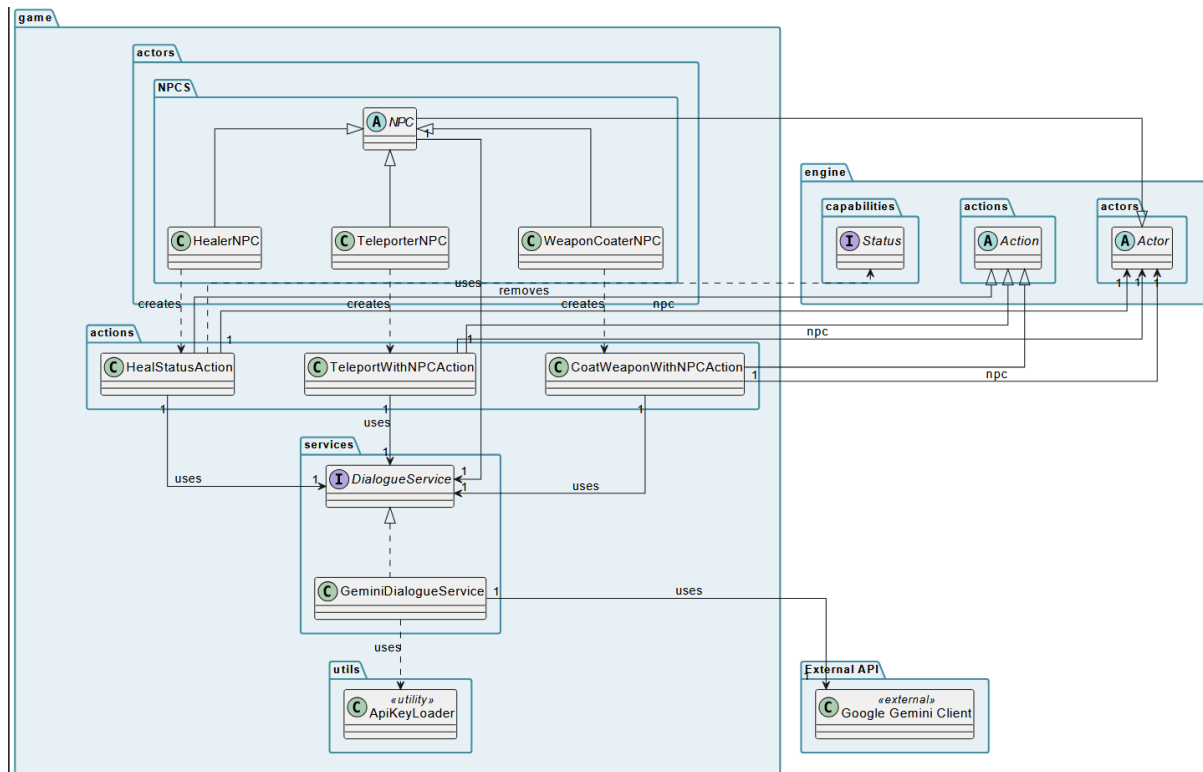


REQ5 (A.P.I) — Design Rationale

NPC Service System with Gemini API Integration

UML Diagram



Design 1: Direct API Integration

In this alternative, each NPC class directly instantiates GeminiDialogueService and manages API calls internally. NPCs have tight coupling to the concrete API implementation.

Advantages:

- Simple and straightforward implementation
- Fewer classes and interfaces
- Direct control over API interactions

Disadvantages:

- **Violates DIP:** NPCs depend on concrete implementation rather than abstraction
- **Difficult Testing:** Cannot substitute mock services; tests require real API calls
- **Poor Extensibility:** Switching to different API provider (OpenAI, Claude) requires modifying all NPC classes

- **Code Duplication:** Each NPC must independently handle API initialization, error handling, and retry logic
- **Violates OCP:** Adding new dialogue providers requires modifying existing NPC code

Design 2: Service Layer with Dependency Injection (Chosen Design)

This design introduces a service layer abstraction separating dialogue generation from NPC behavior. The architecture consists of: **DialogueService** interface defining the contract, **GeminiDialogueService** handling actual API communication, **NPC** abstract base class providing the template, three concrete NPC implementations (TeleporterNPC, WeaponCoaterNPC, HealerNPC), dedicated service actions, and **ApiKeyLoader** utility for secure configuration management.

Connascence Analysis

Connascence of Name enables polymorphism through consistent naming: `getRole()`, `allowableActions()`, `generateGreeting()`, and `generateServiceMonologue()`. All components follow predictable naming conventions.

Connascence of Type is evident in dependencies on abstractions rather than concrete implementations. NPCs depend on `DialogueService` interface, actions receive it through injection, and all components use abstract types from the engine framework, adhering to DIP.

Connascence of Execution Order is critical in action execution flow: dialogue generation must occur before service execution, and API key loading must precede service instantiation. Fallback dialogue only activates when API calls fail.

SOLID Principles Application

Single Responsibility Principle (SRP)

Each component has exactly one responsibility: **ApiKeyLoader** only loads API keys from configuration files; **DialogueService** defines only the dialogue generation contract; **GeminiDialogueService** exclusively handles Gemini API communication and retry logic; each NPC subclass offers one specific service type; and action classes execute one service with dialogue integration. This separation prevents monolithic classes and promotes cohesion.

Open/Closed Principle (OCP)

The **DialogueService** interface allows adding new implementations (OpenAI, Claude, cached service) without modifying the interface or existing code. The **NPC** abstract base class enables creating new NPCs by extension—TeleporterNPC, HealerNPC, and WeaponCoaterNPC were implemented with zero changes to `NPC.java`. Future NPCs like BlacksmithNPC or MerchantNPC can be added without touching existing code.

Liskov Substitution Principle (LSP)

All NPCs can substitute their base types: they can be placed on the map as Actors, call inherited methods correctly, and properly implement required abstract methods. DialogueService implementations are fully substitutable—production code uses GeminiDialogueService while tests use mock implementations, both working identically from the NPC's perspective.

Interface Segregation Principle (ISP)

DialogueService contains only two focused methods that all clients use. This avoids the anti-pattern of a monolithic NPCService interface forcing all implementations to support teleportation, weapon coating, healing, and dialogue generation regardless of actual needs.

Dependency Inversion Principle (DIP)

High-level modules (NPCs) depend on the **DialogueService** abstraction rather than the low-level **GeminiDialogueService** implementation. This enables easy testing (inject mocks as demonstrated in all test files), easy replacement (swap Gemini for other providers), and loose coupling (API changes don't cascade to NPCs).

Advantages

Testability Through Dependency Injection: All NPCs are tested with mock dialogue services (HealerNPCTest, TeleporterNPCTest, WeaponCoaterNPCTest). Tests run without network calls, provide deterministic results, and execute quickly.

Maintainability and Separation of Concerns: Clear boundaries exist between NPC behavior, dialogue generation, and API communication. Each component has well-defined responsibility, making bugs easy to locate and ensuring changes to one component don't cascade to others.

Extensibility Without Modification: New NPCs require only creating a subclass and action—no existing code modification. Swapping dialogue providers requires only implementing the interface. The system demonstrates true OCP compliance.

Security: API keys stored in gitignored local.properties, separated from code. ApiKeyLoader validates keys and provides actionable error messages, following security best practices.

Disadvantages

Increased Structural Complexity: More classes than direct integration (interface + implementation + multiple NPCs + actions). Requires understanding dependency injection and abstraction. **Mitigation:** Comprehensive documentation and clear naming conventions.

API Call Latency: Each service interaction requires ~500-1000ms API call, creating noticeable delay. **Mitigation:** Acceptable for turn-based single-player game; caching possible for high-frequency scenarios.

External Service Dependency: Game functionality partially relies on Gemini API availability. Network issues affect dialogue quality (falls back to generic messages).

Mitigation: Fallback dialogue ensures core functionality always works; retry logic handles transient failures.

Conclusion

This design implements a NPC service system with AI-generated dialogue demonstrating strong SOLID principles adherence. The architecture achieves three working NPCs with distinct services (teleportation, weapon coating, healing), dynamic context-aware dialogue enhancing immersion, graceful error handling preventing crashes, and comprehensive test coverage.